

Reviewer Pack — Minimal Galois Group Order and Circuit Monotone Width Inequa...

Entry ab745c1611ca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 07:46:34 UTC

Minimal Galois Group Order and Circuit Monotone Width Inequality

Entry ID: ab745c1611ca **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 07:46:34 UTC

1. Conjecture

Field A (mathematical branch): Galois Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every Boolean function f with n inputs, the minimal order of a finite group G that can be used to construct a polynomial-time Galois representation for f is upper-bounded by $O(2^{n/\theta(f)})$, where $\theta(f)$ is the circuit monotone width of f .

Rationale (proposer's reasoning):

Galois theory has been successfully applied to study symmetries and invariant properties of functions. The minimal order of a Galois group can be used to represent the symmetry group of a function. Circuit monotone width captures the complexity of circuits that compute the function. A strong relationship between these invariants could reveal new insights into the structure of Boolean functions.

Taxonomy category: Galois_BooleanCircuit (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aa9c22f70ef2c759

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each Boolean function f with n inputs, if the minimal order of Galois group G is less than or equal to $O(2^{n/\theta(f)})$ AND no seed produces a metric greater than 10 for both the minimal order and $\theta(f)$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Galois Group Order AND Circuit Monotone Width - Galois Theory IN BOOLEAN CIRCUIT COMPLEXITY - Order of Finite Groups in Polynomial-Time Representations AND Circuit Monotone Width

Top relevant hits considered: - [<http://arxiv.org/abs/1405.2700v1>] Zero Excess and Minimal Length in Finite Coxeter Groups - [<http://arxiv.org/abs/hep-ph/0610012v1>] Tevatron-for-LHC Report of the QCD Working Group - [<http://arxiv.org/abs/1611.05956v2>] Galois and Cartan Cohomology of Real Groups - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/1903.10609v3>] Circuit Complexity of Knot States in Chern-Simons theory - [<http://arxiv.org/abs/1806.08872v4>] Polynomial-time isomorphism testing of groups of most finite orders - [<http://arxiv.org/abs/1201.0978v2>] A sufficient condition for finite presentability of abelian-by-nilpotent groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_circuit_monotone_width(f):
        n = len(f)
        max_width = 0
        for i in range(1 << n):
            width = 0
            stack = []
            for j in range(n):
                if (i >> j) & 1:
                    stack.append(f[j])
                    width += 1
            else:
                if not stack:
                    continue
                stack.pop()
                width -= 1
```

```

        max_width = max(max_width, width)
    return max_width

def compute_galois_group_order(f):
    n = len(f)
    # Simplified heuristic to estimate the order of the Galois group
    return 2**n

metric_name = "Galois Group Order vs Circuit Monotone Width"
instances_tested = 0
total_order = 0
total_width = 0
n_max = 0

for n in [5, 10, 15, 20, 30, 40]:
    if n > n_max:
        n_max = n

    for _ in range(5): # Test 5 instances per size
        f = generate_random_boolean_function(n)
        order = compute_galois_group_order(f)
        width = compute_circuit_monotone_width(f)

        total_order += order
        total_width += width
        instances_tested += 1

mean_order = total_order / instances_tested
mean_width = total_width / instances_tested

conjecture_holds = mean_order <= (2**n_max / mean_width) and max(mean_order, mean_width) < 10
counterexample = "" if conjecture_holds else f"mean_order={mean_order}, mean_width={mean_width}"

return {
    "metric_name": metric_name,
    "metric_value": mean_order,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's conditions. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14378
2	preregistration	ollama_remote	glm4:latest	0	0	9075
3	novelty	ollama_remote	glm4:latest	0	0	15284
4	novelty	ollama_remote	glm4:latest	0	0	10496
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21557
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10078
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10285
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10468
9	judge	ollama_remote	glm4:latest	0	0	15721

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117342 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/ab745c1611ca.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ab745c1611ca.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ab745c1611ca.tar.gz` (if generated)